

Manual de administración — VM Sonoffice

Este manual explica cómo administrar la instalación de Sonoffice ERP v3.0 en la máquina virtual Ubuntu de la red local. Está pensado para operación diaria: saber dónde está la aplicación, cómo revisar estado, cómo detener/arrancar servicios y cómo desplegar cambios que se trabajan primero en local y se suben a GitHub.

Resumen rápido

Elemento	Valor
Servidor VM	<code>srv-erp-sonoffice</code>
IP LAN	<code>10.16.0.93</code>
Usuario Linux	<code>sonoffice</code>
Sistema operativo	Ubuntu 26.04 LTS
App en servidor	<code>/home/sonoffice/apps/sonoffice</code>
Frontend publicado	<code>/var/www/sonoffice</code>
API	NestJS con PM2, proceso <code>sonoffice-api</code>
Puerto API interno	<code>3001</code>
Acceso web actual	<code>http://10.16.0.93/</code>
Base de datos legacy	MariaDB <code>bd_medios</code> en <code>10.16.0.94:3306</code>
Usuario BD	<code>adminop</code>

Regla clave: `/Erp` es el sistema legacy y se usa como referencia funcional. La nueva implementación vive en `apps/web` y `apps/api`.

Arquitectura de despliegue

Navegador LAN

```
-> http://10.16.0.93/
-> Nginx :80
    -> Frontend estático: /var/www/sonoffice
    -> Proxy /api/* hacia http://127.0.0.1:3001/api/*
        -> API NestJS administrada por PM2
            -> MariaDB legacy 10.16.0.94:3306 / bd_medios
```

Ubicaciones importantes

Ruta	Uso
/home/sonoffice/apps/sonoffice	Repositorio clonado desde GitHub. Aquí se hace <code>git pull</code> , <code>npm ci</code> , <code>npm run build</code> .
/home/sonoffice/apps/sonoffice/.env	Variables de entorno de la API: puerto, BD, credenciales y correo. Permisos <code>600</code> .
/home/sonoffice/apps/sonoffice/apps/api/dist	Build compilado de la API NestJS.
/home/sonoffice/apps/sonoffice/apps/web/dist	Build compilado del frontend React.
/var/www/sonoffice	Carpeta que Nginx sirve al navegador. Se sincroniza desde <code>apps/web/dist</code> .
/etc/nginx/sites-available/sonoffice	Configuración principal de Nginx para la app.
/home/sonoffice/.pm2	Estado y logs de PM2 para procesos Node.

Comandos de estado

Conectarse por SSH a la VM:

```
ssh sonoffice@10.16.0.93
```

Ver estado de la API:

```
pm2 status
```

Ver logs recientes de la API:

```
pm2 logs sonoffice-api --lines 100 --nostream
```

Validar que la API responde y conecta a la base de datos:

```
curl -i http://127.0.0.1:3001/api/health/db  
curl -i http://10.16.0.93/api/health/db
```

Resultado esperado:

```
{"success":true,"data":{"database":"connected"},"message":null}
```

Ver estado de Nginx:

```
systemctl status nginx --no-pager
```

Validar configuración de Nginx:

```
sudo nginx -t
```

Cómo detener y arrancar la API

Detener la API:

```
pm2 stop sonoffice-api
```

Arrancar la API:

```
pm2 start sonoffice-api
```

Reiniciar la API después de cambios:

```
pm2 restart sonoffice-api
```

Guardar el estado de PM2 para que sobreviva reinicios:

```
pm2 save
```

Importante: PM2 ya está configurado con systemd. Si la VM se reinicia, debe restaurar el proceso `sonoffice-api` automáticamente.

Cómo detener y arrancar el frontend

El frontend no corre como proceso Node en producción. Es estático y lo sirve Nginx desde `/var/www/sonoffice`.

Para "detener el frontend", se detiene Nginx:

```
sudo systemctl stop nginx
```

Para volver a publicarlo:

```
sudo systemctl start nginx
```

Para recargar Nginx después de cambiar configuración:

```
sudo systemctl reload nginx
```

Para reiniciar Nginx completo:

```
sudo systemctl restart nginx
```

Paso a paso para implementar ajustes desde Git en la VM

Este es el flujo operativo recomendado cuando un ajuste ya fue desarrollado en local, probado, commiteado y subido a GitHub. La idea es simple: **la VM no se usa para programar; la VM solo descarga código validado, compila, publica el frontend y reinicia la API.**

Regla de oro: si un comando falla, NO sigas con el siguiente "a mano". Copiá el error, entendé la causa y recién ahí continuá. En producción, improvisar es como cambiar una viga sin mirar el plano: puede quedar parado... hasta que no.

Resumen del flujo

Etapas	Dónde se hace	Objetivo
Validar cambios	Máquina local	Confirmar que el código compila antes de subirlo.
Subir a GitHub	Máquina local	Dejar el cambio disponible para la VM.
Actualizar repo	VM 10.16.0.93	Descargar la última versión con <code>git pull</code> .
Compilar	VM	Generar <code>dist</code> de API y frontend.
Publicar frontend	VM	Copiar <code>apps/web/dist</code> a <code>/var/www/sonoffice</code> .
Reiniciar API	VM	Recargar el proceso <code>sonoffice-api</code> en PM2.
Validar	VM y navegador	Confirmar API, base de datos y web.

1. Validar y subir el cambio desde la máquina local

Entrar al repo local:

```
cd /Users/josenarvaez/Projects/sonoffice
```

Revisar qué archivos cambiaron:

```
git status --short --branch
```

Validar TypeScript y compilación completa:

```
npm run typecheck  
npm run build
```

Resultado esperado:

- `npm run typecheck` termina sin errores.
- `npm run build` termina sin errores.
- Puede aparecer un warning de Vite por tamaño de bundle; ese warning no bloquea si el build finaliza correctamente.

Revisar el diff antes de commitear:

```
git diff --stat  
git diff --check
```

Crear commit con los archivos del ajuste:

```
git add <archivos-cambiados>
git commit -m "feat(scope): descripcion corta"
```

Ejemplo real:

```
git add apps/api/src/cost-orders apps/web/src/pages/CostOrderForm.tsx
git commit -m "feat(cost-orders): support budget lines during order
creation"
```

Subir a GitHub:

```
git push origin main
```

Verificar que local quedó limpio:

```
git status --short --branch
```

Resultado esperado:

```
## main...origin/main
```

No despliegues en la VM si `npm run build` falla en local. La VM debe recibir código ya validado.

2. Conectarse a la VM

Desde la máquina local:

```
ssh sonoffice@10.16.0.93
```

Confirmar que estás en la VM correcta:

```
hostname
pwd
```

Resultado esperado:

```
srv-erp-sonoffice
/home/sonoffice
```

Entrar al repositorio clonado:

```
cd /home/sonoffice/apps/sonoffice
```

Confirmar rama y estado antes de tocar nada:

```
git status --short --branch
```

Resultado esperado antes del pull:

```
## main...origin/main [behind 1]
```

También puede aparecer simplemente:

```
## main...origin/main
```

Si aparecen archivos modificados en la VM, por ejemplo:

```
M apps/api/src/...  
?? archivo-temporal.txt
```

detenete. Eso significa que alguien cambió archivos directamente en la VM o quedaron residuos. No hagas `git reset --hard` sin revisar, porque podrías borrar información útil. Primero inspeccioná:

```
git diff --stat  
git diff
```

3. Descargar la última versión desde GitHub

Antes de traer cambios, confirmar que la VM no tenga modificaciones locales:

```
git status --short --branch
```

Si aparece solo la rama, podés seguir:

```
## main...origin/main
```

Si aparece `package-lock.json` modificado, normalmente fue generado por `npm install` en la VM con una versión distinta de npm:

```
M package-lock.json
```

No lo committees desde la VM. Limpiá ese cambio antes del pull:

```
git checkout -- package-lock.json
git status --short --branch
```

Si aparecen otros archivos modificados, no los borres a ciegas. Revisá primero:

```
git diff --stat
git diff
```

Traer cambios solo si la VM puede avanzar en línea recta:

```
git pull --ff-only origin main
```

Resultado esperado:

```
Updating <commit-anterior>..<commit-nuevo>
Fast-forward
...
```

Si dice `Already up to date`, la VM ya tiene el último código.

Si falla por cambios locales, NO fuerces. Revisá `git status --short` y resolvé la causa.

Confirmar el commit desplegado:

```
git log --oneline -1
```

Debe mostrar el último commit que se subió desde local.

4. Instalar dependencias sin reescribir el lockfile

En la VM usá `npm ci`, no `npm install`:

```
npm ci
```

`npm ci` instala exactamente lo definido en `package-lock.json` y evita que la VM lo reescriba. Esto reduce problemas en futuras actualizaciones con `git pull`.

Este paso es obligatorio si cambiaron:

- `package.json`
- `package-lock.json`
- dependencias de `apps/api`
- dependencias de `apps/web`

Si no cambiaron dependencias, igual se puede ejecutar; solo tardará un poco más.

Importante: `npm install` es para desarrollo local cuando se agregan o actualizan paquetes. En despliegue de VM, preferí `npm ci` para una instalación reproducible.

5. Compilar API y frontend en la VM

Ejecutar build completo:

```
npm run build
```

Este comando hace dos cosas:

1. Compila la API NestJS en `apps/api/dist`.
2. Compila el frontend React/Vite en `apps/web/dist`.

Resultado esperado:

- No hay errores TypeScript.
- Se genera/actualiza `apps/api/dist`.
- Se genera/actualiza `apps/web/dist`.

Si falla el build:

1. No reinicies PM2.
2. No copies el frontend.
3. Copiá el error completo.
4. Corregí en local, hacé commit, push y recién volvé a desplegar.

6. Publicar el frontend en Nginx

El frontend que ve el usuario NO sale directamente de `apps/web/dist`; Nginx sirve la carpeta:

```
/var/www/sonoffice
```

Por eso, después de compilar, hay que copiar el build nuevo:

```
sudo rsync -a --delete /home/sonoffice/apps/sonoffice/apps/web/dist/  
/var/www/sonoffice/
```

Luego asegurar permisos correctos para Nginx:

```
sudo chown -R www-data:www-data /var/www/sonoffice
```

Validar que el frontend quedó publicado:

```
ls -la /var/www/sonoffice
```

Debe verse, como mínimo:

```
index.html  
assets/
```

El `--delete` de `rsync` es intencional: borra assets viejos que ya no existen en el build nuevo. Sin eso, el navegador podría cargar archivos antiguos.

7. Reiniciar la API con PM2

Ver estado antes de reiniciar:

```
pm2 status
```

Reiniciar la API:

```
pm2 restart sonoffice-api
```

Guardar el estado de PM2:

```
pm2 save
```

Verificar que quedó online:

```
pm2 status
```

Si aparece `errored`, revisar logs:

```
pm2 logs sonoffice-api --lines 100 --nostream
```

8. Validar API, base de datos y web

Validar API local en la VM:

```
curl -i http://127.0.0.1:3001/api/health/db
```

Validar API pasando por Nginx:

```
curl -i http://10.16.0.93/api/health/db
```

Resultado esperado en ambos casos:

```
{"success":true,"data":{"database":"connected"},"message":null}
```

Validar que la web responde:

```
curl -I http://10.16.0.93/
```

Resultado esperado:

```
HTTP/1.1 200 OK
```

Abrir en navegador:

```
http://10.16.0.93/
```

Probar manualmente:

- Login con usuario real.
- Menú lateral.
- Pantalla modificada por el ajuste.
- Una operación básica del flujo cambiado.

Ejemplo: si el cambio fue en Órdenes de Costo, validar listado, creación/edición, mensajes, impresión o asociación con presupuesto según corresponda.

9. Checklist final de despliegue

- ☐ El commit correcto está en GitHub.
- ☐ La VM está en `/home/sonoffice/apps/sonoffice`.

- ☐ `git pull --ff-only origin main` terminó bien.
- ☐ `git log --oneline -1` muestra el commit esperado.
- ☐ `npm ci` terminó bien.
- ☐ `npm run build` terminó bien.
- ☐ `rsync` copió `apps/web/dist` a `/var/www/sonoffice`.
- ☐ `pm2 restart sonoffice-api` terminó bien.
- ☐ `pm2 status` muestra `sonoffice-api` online.
- ☐ `/api/health/db` devuelve `database: connected`.
- ☐ `http://10.16.0.93/` abre en navegador.
- ☐ Se probó con usuario real el flujo afectado.

Comando compacto de despliegue en VM

Usá este comando solo cuando el cambio ya está en GitHub y querés actualizar la VM siguiendo el camino feliz:

```
cd /home/sonoffice/apps/sonoffice \  
&& git status --short --branch \  
&& git pull --ff-only origin main \  
&& git log --oneline -1 \  
&& npm ci \  
&& npm run build \  
&& sudo rsync -a --delete apps/web/dist/ /var/www/sonoffice/ \  
&& sudo chown -R www-data:www-data /var/www/sonoffice \  
&& pm2 restart sonoffice-api \  
&& pm2 save \  
&& pm2 status \  
&& curl -i http://127.0.0.1:3001/api/health/db \  
&& curl -I http://10.16.0.93/
```

Si algún paso falla, el `&&` corta la ejecución. Eso es bueno: evita publicar una app a medias.

Variables de entorno

El archivo `.env` vive en:

```
/home/sonoffice/apps/sonoffice/.env
```

Valores principales actuales:

```
PORT=3001
DB_HOST=10.16.0.94
DB_PORT=3306
DB_NAME=bd_medios
DB_USER=adminop
BREVO_SENDER_EMAIL=sistemaop@sonovista.co
BREVO_SENDER_NAME=Sonoffice
```

No imprimir ni compartir `DB_PASSWORD` ni `BREVO_API_KEY`.

Para revisar sin exponer secretos:

```
grep -E "^(PORT|DB_HOST|DB_PORT|DB_NAME|DB_USER|BREVO_SENDER)"
/home/sonoffice/apps/sonoffice/.env
```

Nginx

Archivo de configuración:

```
/etc/nginx/sites-available/sonoffice
```

Validar y recargar:

```
sudo nginx -t
sudo systemctl reload nginx
```

El server block actual debe servir:

- `/` desde `/var/www/sonoffice`
- `/api/` hacia `http://127.0.0.1:3001/api/`

Cuando se cree DNS local, agregar el dominio en `server_name`, por ejemplo:

```
server_name sonoffice.lan 10.16.0.93 srv-erp-sonoffice;
```

DNS local pendiente

Hoy el acceso es por IP:

```
http://10.16.0.93/
```

Para usar nombre en toda la red, crear en el DNS interno `10.16.0.14` un registro A:

```
sonoffice.lan -> 10.16.0.93
```

Después validar:

```
nslookup sonoffice.lan 10.16.0.14
curl -I http://sonoffice.lan/
```

Troubleshooting rápido

Problema	Validar	Acción típica
No abre la web	<code>systemctl status nginx --no-pager</code>	<code>sudo systemctl restart nginx</code>
Web abre pero API falla	<code>pm2 status</code>	<code>pm2 restart sonoffice-api</code>
API no conecta a BD	<code>curl http://127.0.0.1:3001/api/health/db</code>	Revisar <code>.env</code> , red y acceso a <code>10.16.0.94:3306</code>
Cambios frontend no aparecen	Revisar <code>/var/www/sonoffice</code>	Rehacer <code>npm run build + rsync</code>
Pull falla por cambios locales en VM	<code>git status --short</code>	No pisar a ciegas; revisar qué archivo cambió
Nginx no recarga	<code>sudo nginx -t</code>	Corregir sintaxis antes de reload

Checklist después de cada despliegue

- ☐ `git pull --ff-only origin main` terminó sin errores.
- ☐ `npm ci` terminó sin errores.
- ☐ `npm run build` terminó sin errores.
- ☐ Frontend sincronizado a `/var/www/sonoffice`.
- ☐ `pm2 restart sonoffice-api` ejecutado.
- ☐ `curl http://127.0.0.1:3001/api/health/db` devuelve `database: connected`.

- ☐ `http://10.16.0.93/` abre en navegador.
- ☐ Login y menú lateral probados con usuario real.